

Operating Systems (Honor Track)

timetraveler314

University of Genshin

timetraveler314@outlook.com

1. Abstraction

2. Synchronization

3. Scheduling

3.1. Concepts and Classic Policies

3.1.1. Scheduling Policy Goal/Criteria

Without any a priori knowledge of the workload, we cannot design a scheduling algorithm that is optimal for all possible workloads. Instead, we can only aim to design algorithms that are optimal for certain classes of workloads. Hence different criteria are used to evaluate the performance of scheduling algorithms. The high-level goal: dole out CPU time to optimize some desired performance metric. The most common criteria are:

- **Minimize Completion Time:** Minimize the time from the submission of a job to the time of its completion.
 - Real-time tasks: must meet their deadlines.
 - Overhead: more context switches than only maximizing throughput.
- **Maximize Throughput:** Maximize the number of jobs completed in a given time period.
 - Minimize overhead, or
 - Efficiently utilize the resources.
- **Fairness:** Ensure that all jobs get a fair share of the CPU.
 - Fairness is not about minimizing average completion time, but
 - Better average completion time can be achieved by making system less fair.

3.1.2. FCFS Scheduling

Pros and cons of FCFS scheduling:

- (+) Simple.
- (-) **Head-of-line (HoL) blocking:** short process waiting for long process.

3.1.3. RR Scheduling

Round Robin scheme: **preemption!** CPU time is divided into small time slices (quantum), and each process is given a time slice in which to execute. If the process does not finish within its time slice, it is preempted and put back in the queue.

Performance evaluation:

- Large $q \Rightarrow$ FCFS
- Small $q \Rightarrow$ Interleaving, but overhead
- Hence q must be large enough to amortize the overhead of context switching. Typical values are 10-100ms.

Pros and cons of RR scheduling:

- (+) Fairness: better for short jobs.

- (-) Overhead: context switching adds up for long jobs.

Implementation:

3.1.4. What if We Knew the Future? SJF and SRTF Scheduling

- **Shortest Job First (SJF)** scheduling: the process with the smallest estimated time to completion is scheduled next. This is a non-preemptive algorithm.
- **Shortest Remaining Time First (SRTF)** scheduling: preemptive version of SJF. If a job arrives with a shorter time to completion than the currently running job, the current job is immediately preempted and the new job is scheduled.

Knowing the future, they are **Provably Optimal**:

- SJF and SRTF are optimal for minimizing average completion time (among all non-preemptive and preemptive algorithms, respectively).

Further discussion

- Starvation: when there are many short jobs, long jobs may never get scheduled.
- No future knowledge: SJF and SRTF are not implementable in practice.
 - But can be used as yardstick to evaluate other algorithms.

SRTF Pros & cons:

- (+) Optimal in terms of average completion time
- (-) Hard to predict the future
- (-) Unfair

Adaptive Prediction: changing policy on past behavior. This can be used to implement SRTF with estimated burst length: for instance, *exponential averaging*:

$$\tau_n = \alpha t_{n-1} + (1 - \alpha)\tau_{n-1}.$$

3.1.5. Lottery Scheduling

- On each time slice, random pick a winning ticket $\Rightarrow$ On average, CPU time is proportional to #tickets.
- **How to assign?**
 - To approximate SRTF, more for short-running jobs
 - To avoid starvation, every job gets at least one
- Advantage over (strict) priority scheduling: behave gracefully as load changes

3.1.6. Multi-Level Feedback Scheduling - exploiting past behavior

Key:

- Multiple queues, each with different priority
- Each queue has its own scheduling algo:
 - E.g. foreground RR (for responsive jobs), background FCFS
 - Also: multiple RR, with quantum increasing exponentially. (Highest 1ms, next 2ms, next 4ms, etc.)
- **Feedback:** adjust each job's priority based on its behavior
 - Job starts in the highest priority queue
 - If timeout expires (which indicates that the job is CPU-bound), demote it to a lower-priority queue
 - If job yields before timeout, promote it to a higher-priority queue (or to top)

3.2. Case Studies: Fairness, Real Time, and Forward Progress

3.3. Scheduling and Deadlock

Theorem 3.1.

- **Mutual exclusion:** At least one resource must be held in a non-shareable mode.
- **Hold and wait:** A process holding at least one resource is waiting to acquire additional resources that are currently being held by other processes.
- **No preemption:** Resources cannot be forcibly taken from a process holding them, but can only be released voluntarily by the process after it has completed its task.
- **Circular wait:** Formally, a set of threads $T_1, T_2, \dots, T_n$ is in a circular wait state if T_1 is waiting for a resource held by T_2 , T_2 is waiting for a resource held by T_3 , and so on, until T_n is waiting for a resource held by T_1 .

3.3.1. Detecting Deadlock

We use the following resource allocation model:

- There is a set of threads $T_1, T_2, \dots, T_n$ and a set of resources $R_1, R_2, \dots, R_m$.
- Each resource R_i has a number of instances W_i .
- *Resource allocation graph:* a directed graph with nodes representing threads and Resources
 - request edge: $T_i \rightarrow R_j$ if T_i is waiting for R_j
 - assignment edge: $R_j \rightarrow T_i$ if T_i is holding R_j
 - Somewhat counterintuitive, but: a finished thread will assign its resources by freeing the “source” of the edge. “ $\rightarrow$ ” is actually “depending on”.

The Worklist Algorithm (let $[x]$ represents a m -ary vector, where m is the number of kinds of resources):

```
[Avail] = [FreeResources]
Add all nodes to UNFINISHED set
do {
    changed = false
    for each node T_i in UNFINISHED {
        if ([Request_i] <= [Avail]) { // T_i can progress
            changed = true
            UNFINISHED.remove(T_i)
            [Avail] += [Allocation_i] // T_i releases its resources
        }
    }
} while (changed); // repeat until no changes

if UNFINISHED is empty {
    // No deadlock
} else {
    // Deadlock
}
```

4. Virtual Memory

4.1. Caching and TLBs

CTag CIndex CO

Note. Sources of Cache Misses

- **Compulsory** (cold start)
- **Capacity** (cache size)
- **Conflict** (cache associativity, collisions)
- **Coherence** (invalidation, e.g. in multiprocessor systems)

• **Physically-Indexed vs Virtually-Indexed Caches:**

- Physically-indexed
 - Challenge: TLB is on the critical path
 - Solution: TLB is virtually-indexed, but physically-tagged
- Virtually-indexed
 - Challenge: Same data may be mapped in many places of the cache, may need to flush the cache on context switch

4.1.1. TLB Organization

- Choice of Index: trade-off in thrashing
 - Using low-order bits as index: first pages of code, stack and heap are likely to be mapped to the same cache line, requiring greater associativity.
 - Using high-order bits as index: insufficient TLB utilization, as small programs tend to have sparse address space.

4.2. Demand Paging

4.2.1. Eviction Policies

- **Least Recently Used (LRU)**: evict the page that has not been used for the longest time.
 - Performance guarantee? The classic sequential $N + 1$ working set on N frames leads to each access being a page fault.
 - However, the follow *stack property* holds:

Theorem 4.1. Stack Property of LRU and MIN

When you increase memory size, the miss rate should never increase. This holds for both LRU and MIN.

Equivalently, the set of pages in memory for size n is always a subset of the pages in memory for size $n + 1$.

- **Clock Algorithm**: Familiar.
- **Second Chance List** on VAX arch (without hardware support):
 - Two lists
 - Active: access at full speed
 - SC: access intercepted by OS page fault handler
 - Accessing a page in SC moves it to Active.
 - On eviction, the last page in SC is evicted.
- **Free List** speeding up PF handler:
 - A pageout daemon selects pages to fill the free list.
 - Upon entering the free list, the dirty pages start to be written back to disk.
 - When a page is needed, it is directly taken from the free list, reducing the overhead of page fault handling.

4.2.2. Coremap (Reverse Mapping)

To correctly invalidate all the PTEs related to a frame, we need to maintain a reverse mapping from frames to PTEs. This is called the coremap.

A per-frame structure costs too much, so a coarse-grained structure is used instead in Linux. The frame metadata are recorded by the Object-based reverse mapping, per segment.

4.2.3. Allocation of Frames across Processes

- Fixed Scheme: Equal/Proportional (to size) allocation of frames across processes.
- Page fault frequency allocation: establish “acceptable” page-fault rate for each process, and allocate frames dynamically to maintain that rate.
 - *Thrashing* if we just don’t have enough frames. Best response? Working set model may help.
- **Working Set Model:**
 - Working Set defines minimum number of pages for process to behave well, since processes exhibit locality of reference.
 - *Working Set Window* Δ : a fixed number of page references.
 - In the window, the pages that are referenced are the working set WS_i for P_i .
 - *System Demand* $D = \sum_i |WS_i|$: the total number of pages in the working sets of all processes.
 - if $D > M$, then the system is thrashing.
 - Policy: **swap out** process at this point to guarantee overall system performance!

Note. Compulsory Misses and Working Set

- **Clustering:** Bring multiple neighboring pages into memory at a page fault.
 - Since efficiency of disk reads increases with sequential reads, makes sense to read several sequential pages
- **Tracking Working Set:**
 - A swapped-out process’s working set is recorded, and when it is swapped back in, the pages are prefetched into memory.

4.2.4. Memory Management in Modern Computer Systems

4.2.4.1. FaRM: Fast Remote Memory

- Hardware trends:
 - Memory is getting cheaper, and hence larger.
 - Network bandwidth is increasing, with RDMA clusters.
- FaRM:
 - Keep data in memory, and use RDMA to access it.
 - Lock-free reads
 - Transactional updates
 - Colocate data with computation.
 - Colocate data accessed together
 - Ship computation to data (RPC)
 - Optimized single-server transactions
 - Symmetric model with a shared address space.
 - Transparent to applications, w.r.t location, concurrency, and failure.

4.2.4.2. vLLM

- **Resource Preemption and Recovery:**

- Difference from OS VM: recomputation is surprisingly fast since no decoding is needed, and prefill can be done in parallel. So recovery is based on recomputation.

4.2.4.3. Infiniswap: Efficient Memory Disaggregation

- Background: memory-intensive applications. How to solve the memory underutilization problem?
- Infiniswap: disaggregate memory across machines.
 - Remote paging: no hardware design, no application modification.
 - Local backup disk: fault-tolerant.
 - **Decentralized Memory Management**: scalable
 - *Random Load Balancing* with the Power of Two Choices: each node randomly selects two remote nodes to swap with, and chooses the one with the least memory usage.

4.2.4.4. AIFM: Application-Integrated Far Memory

AIFM balances the trade-off between performance and transparency.

- **Semantic Gap**: Bridged by the remoteable data structure library, enabling prefetch, etc.
- **Kernel Overhead**: Reduced by user-space page swapping, which allows the application to handle page faults directly.
- **Memory Reclamation**: Utilizes a *pauseless* evacuator
- **Network Bandwidth**: Remote agent to conduct remote memory operations

4.2.4.5. PipeSwitch: Fast Pipelined Context Switching for Deep Learning Applications

- Background: dedicated clusters for training/inference led to underutilization of resources, and current context switching is too slow.
- Reduced the overhead in:
 - Model Transmission: pipelined transmission of model weights and execution, utilizing the layer-wise structure of the model.
 - Memory Management: unified, with a daemon providing pointer to workers
 - Task initialization and cleaning: overlap them by active-standby worker switching
 - Initialization: two stages
 - Process and CUDA context creation: may overlap with the old task
 - GPU memory allocation: overlap with the cleanup of the old task

4.2.4.6. TGS: Transparent GPU Sharing in Container Clouds for Deep Learning Workloads

OS-layer, transparency oriented GPU sharing. Allowing for oversubscription of GPU resources in container clouds.

- **DL Jobs**: Production jobs and Opportunistic jobs
- Key ideas: leverage CUDA unified memory to transparently unify GPU memory and host memory
 - GPU utilization and fault isolation (eviction of opportunistic jobs)

5. File Systems

5.1. IO Performance and File System Design

5.1.1. Queuing Theory

Overall, the Markovian model is used to analyze the performance of queuing systems, which characterizes the arrival process as a Poisson process. As for service time, it is often modeled as an exponential distribution, but can also be generalized to other distributions, characterized by their C :

$$C = \frac{\sigma(\text{Service Time})^2}{\mathbb{E}[\text{Service Time}]^2}$$

- $C = 0$ for deterministic service time
- $C = 1$ for exponential service time, memoryless

System parameters:

- λ : arrival rate
- μ : service rate
- $u(\rho)$: utilization, $u = \frac{\lambda}{\mu} = \lambda \cdot T_{\text{service}}$

System performance metrics:

- T_q : average time spent in the queue
- L_q : average number of jobs in the queue

Theorem 5.1. Results of Queuing System

The discussion is based on Poisson arrivals and 1 server:

- **M/M/1** queue (memoryless, $C = 1$):

$$T_q = T_{\text{service}} \cdot \frac{u}{1 - u}.$$

- **M/G/1** queue (general service time, $C > 0$):

$$T_q = T_{\text{service}} \cdot \frac{1 + C}{2} \cdot \frac{u}{1 - u}.$$

Theorem 5.2. Little's Law

For any stable queuing system, the following relationship holds: $L = \lambda \cdot T$, where L is the average number of jobs in the system, λ is the arrival rate, and T is the average time spent in the system.

5.1.2. Disk Scheduling

- **SSTF: Shortest Seek Time First**: choose the request with the shortest *seek + rotational latency* (since rotation can be as long as seek time).
 - Pros: reduces average seek time
 - Cons: starvation for requests far away from the current head position
- **SCAN**: the disk arm moves in one direction, servicing the nearest requests until it reaches the end of the disk, then reverses direction.
 - Pros: avoids starvation, remains some flavor of SSTF
- **C-SCAN**: Circular SCAN, only goes in one direction.
 - Pros: Fairer than SCAN, not biased towards pages in middle

5.2. File System Case Studies, Buffering

5.2.1. FAT: File Allocation Table

Simply put, FAT is just a linked list of blocks, with the first block being the file number.

- **Directories**: A file of name $\rightarrow$ file number mapping. Organized as a linked list of directory entries, each containing the file name, file number, and other metadata.
 - *Attributes*: file attributes are kept in directory

- *Root*: always at block 2.

5.2.2. FFS: Unix BSD Fast File System

- **Inodes**: Each file has an inode, which contains metadata (unlike FAT) about the file, including its size, permissions, and pointers to the data blocks.
 - **Inumber** is used to index into the inode table
 - Pointers:
 - 12 direct pointers to data blocks
 - 1 single indirect pointer to a block containing pointers to data blocks
 - 1 double indirect pointer to a block containing pointers to single indirect blocks
 - 1 triple indirect pointer to a block containing pointers to double indirect blocks
- **Pros**:
 - Efficient storage for both small and large files
 - Locality for both small and large files
 - Locality for metadata and data
 - Data blocks, metadata, and free space interleaved within **block group** (set of tracks close in distance)
 - No defragmentation necessary!
- **Cons**:
 - Inefficient for tiny files (a 1 byte file requires both an inode and a data block)
 - Inefficient encoding when file is mostly contiguous on disk (i.e. the interleaving)
 - Need to reserve 10-20% of free space to prevent fragmentation

5.2.3. Links

- **Hard**: Extra File Name → File Number mapping. Reference count in inode.
 - `link()` and `unlink()` system calls.
- **Soft**: Symbolic, File Name → Dest. File Name
 - `symlink()` system call.

5.2.4. Directory Traversal

- **Linear Search**: When organized as a linked list, the only feasible approach is linear search. Simple, but slow.
- **B-Tree**: Indexed by hash.

5.2.5. Windows NTFS

Organized with **Master File Table (MFT)**.

- File: Each file has an entry in the MFT, which contains metadata and data:
 - Either the data itself (esp. small files)
 - or List of **Extents**: contiguous blocks of data on disk, providing variable length allocation;
 - or even pointers to other MFT entries with more extent lists (for large files).
- Directory: B-trees.

5.3. Buffering, Reliability, and Transactions